

MVC e Padrões de Projeto

Notas organizadas a partir dos slides e das anotações manuscritas de aula, Setembro/2026.

Disciplina: Métodos Avançados de Programação (MAP) — UEPB, 26.2

Professora: Luciana de Queiroz Leal Gomes

Referências: Larman (Utilizando UML e Padrões) · GoF (Padrões de Projeto) · Freeman & Freeman (Use a Cabeça! Padrões de Projetos) · Bloch (Java Efetivo)

Material Auxiliar

Métodos Avançados de Programação

MVC como Composição de Padrões de Projeto

1. Objetivos da aula

- Entender o **papel do MVC**.
- Apresentar o uso de **alguns padrões clássicos no MVC**.
- Compreender a **separação de responsabilidades**.

- Reconhecer esses conceitos na prática.

2. O que é o MVC?

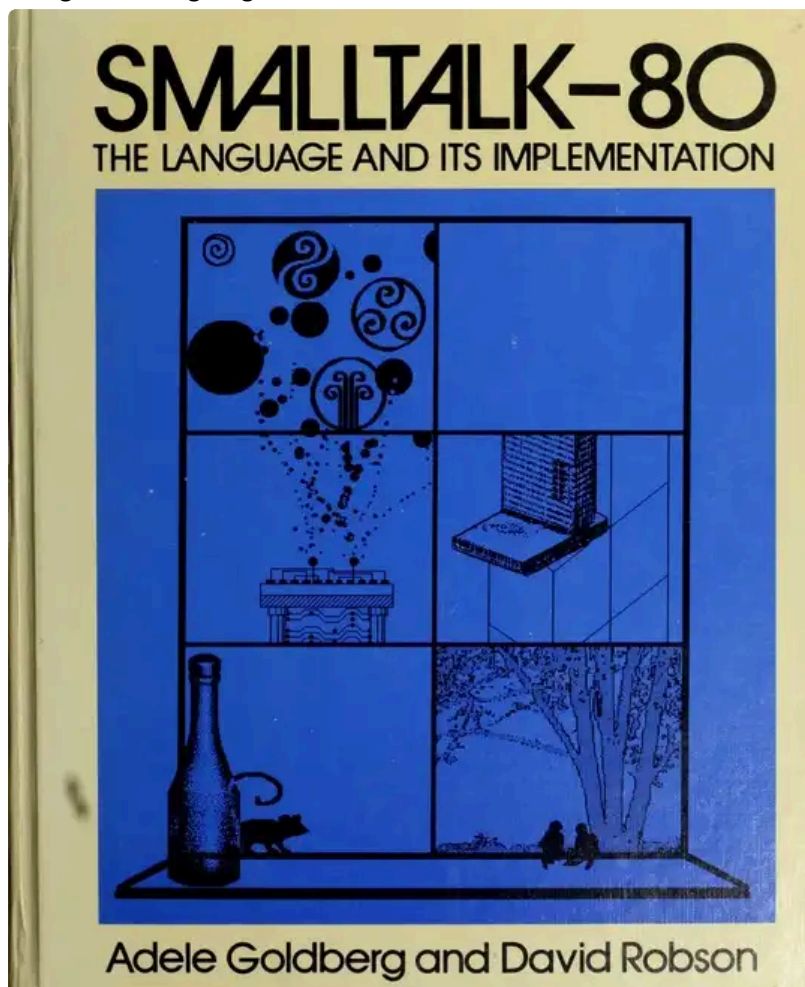
2.1 Padrão ou arquitetura?

O MVC é **um único padrão** ou **uma combinação de padrões**? A tese da aula é que o MVC é um **padrão arquitetural em camadas** que se apoia em uma **combinação de padrões de projeto**.

As camadas **se conhecem em parte**: o Controller conversa com a View e com o Model, e a View *observa* o Model. O que **não** acontece é o Model conhecer a View. A ideia correta é "camadas com **baixo acoplamento** e responsabilidades separadas".

2.2 Origem e uso

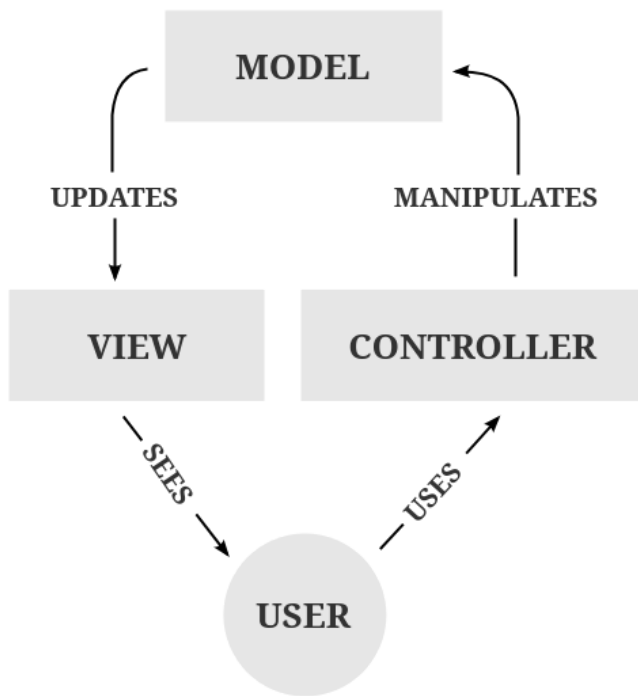
- Surgiu na linguagem Smalltalk.



- É popular em frameworks web (Django, Spring, etc.).
- É um dos primeiros padrões identificados.

2.3 Definição e propósito

MVC = Model-View-Controller.



- Padrão **arquitetural em camadas**.
- **Separa a lógica de negócio da interface**.
- Possui **uma camada que controla as requisições** realizadas.
- Procura resolver o **acoplamento excessivo** entre interface gráfica e lógica da aplicação.

Se os dois estiverem muito acoplados, **o desenvolvimento e a manutenção ficam difíceis**. O MVC "melhora o modelo de 2 camadas cliente-servidor".

Problemas que o MVC ataca

Quando a interface está acoplada à lógica, surgem quatro problemas :

1. **Interfaces acopladas à lógica.**
2. **Dificuldade de manutenção.**
3. **Código difícil de reutilizar.**
4. **Mudanças quebram outras partes.**

São as consequências do **alto acoplamento**

3. Os três componentes do MVC

3.1 Definições

Model (dados)

Camada responsável pela **lógica de negócios, manipulação de dados e regras do sistema**. 📝 Exemplos no iFood: usuário, pedido, restaurante, entregador, pagamento.

View (interface)

Camada de **interface do usuário**, responsável pela **apresentação dos dados e pela interação**. É onde o usuário vê o estado do modelo e **manipula a interface para ativar a lógica de negócio**.

Controller

Camada **intermediária** responsável por **receber as requisições do usuário, processar a lógica de entrada e coordenar a interação entre o Model (dados) e a View (interface)**. Também é descrito como quem **transforma eventos gerados pela interface em ações de negócio, alterando o modelo**.

O MVC faz algo já discutido antes: **separação de responsabilidades**.

3.2 Objetivo do MVC

- Separar **dados ou lógica de negócios (Model)** da **interface do usuário (View)** e do **fluxo da aplicação (Controller)**.
- Permitir que **uma mesma lógica de negócios** seja **acessada e visualizada por várias interfaces**.
- Na arquitetura MVC, **a lógica de negócios (o Modelo) não sabe quantas nem quais interfaces** estão exibindo seu estado.

3.3 Fluxo do MVC

1. O usuário **interage com a View**.
2. O **Controller interpreta a ação**.
3. O **Controller atualiza o Model**.
4. O **Model notifica a View**.
5. A **View se atualiza**.

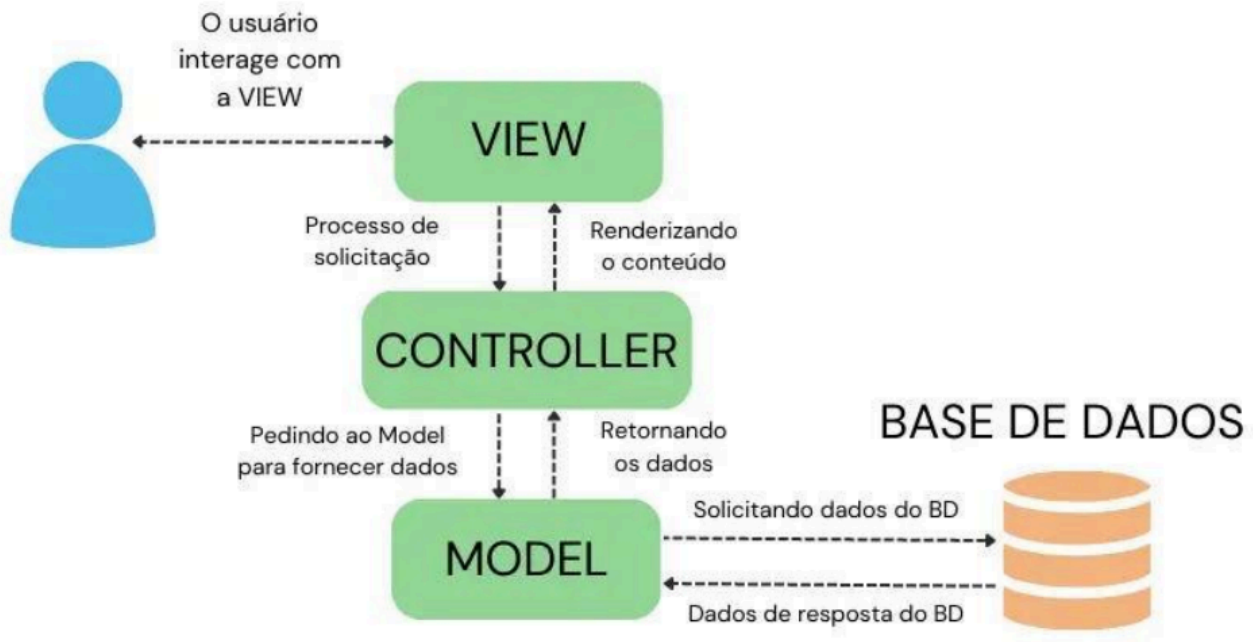
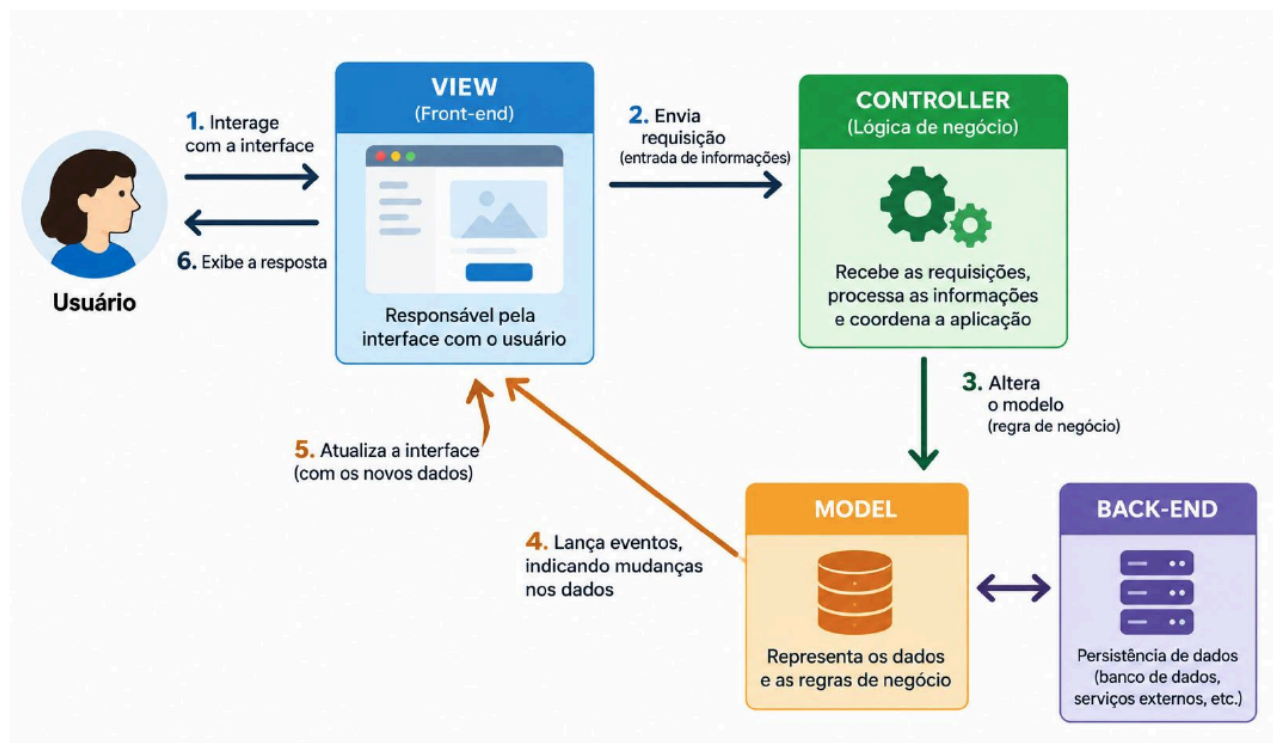


Diagrama "Solução com MVC" (6 passos)



1. O **usuário interage com a interface** (View, front-end).
2. A View **envia a requisição** (entrada de informações) ao Controller.
3. O Controller **altera o modelo** (regra de negócio).
4. O Model **lança eventos, indicando mudanças nos dados**.
5. A View **atualiza a interface** com os novos dados.
6. A View **exibe a resposta** ao usuário.

O Model representa os dados e as regras de negócio e se liga ao **back-end** (persistência: banco de dados, serviços externos, etc.).

⚠ Há duas inconsistências entre as figuras e o texto dos slides da professora. **(1)** Na figura de 6 passos, o quadro do Controller está rotulado "**Lógica de negócio**", mas o texto define a lógica de negócio como papel do **Model**; o Controller trata da **lógica de entrada e da coordenação**. **(2)** O texto diz "Model notifica a View" (passo 4 do fluxo), enquanto a figura com base de dados faz a View receber o conteúdo **via Controller**. Versão adotada aqui: Model = negócio e dados; Controller = coordena; a atualização automática da View acontece por **notificação do Model (Observer)**. O diagrama das anotações também **não desenha** a seta Model → View; ela existe pelo Observer.

4. Estudo de caso: sistema tipo iFood



4.1 O cenário simplificado

Modelagem de um sistema com:

- **Usuário faz um pedido.**
- **Restaurante prepara.**
- **Sistema escolhe entregador.**
- **Pagamento é processado.**
- **Usuário recebe atualizações do pedido.**

"Como você faria?"

4.2 Solução sem MVC

- **Onde está a lógica de negócio?**
- **Onde está a parte gráfica?**
- **Há acoplamento?**

- A parte gráfica existe sem a parte da lógica?

Sem MVC, lógica e parte gráfica tendem a ficar misturadas; assim, uma não existe sem a outra (acoplamento).

4.3 Solução com MVC

A aplicação é dividida em **três partes**:

1. **MODEL**: lógica de negócio.
2. **VIEW**: camada de interface com o usuário; o usuário vê o estado do modelo e manipula a interface para ativar a lógica de negócio.
3. **CONTROLLER**: transforma eventos gerados pela interface em ações de negócio, alterando o modelo.

4.4 Da arquitetura à implementação

- A solução MVC apresenta a **arquitetura do sistema (alto nível)**.
- Pergunta: **de que forma desacoplar a nível de implementação?** Resposta: por meio de **boas práticas de desenvolvimento**.
- Algumas dessas práticas estão catalogadas como **padrões de projeto**.

5. Padrões de projeto



5.1 O que são e de onde vêm

O que são:

- **Soluções recorrentes** para problemas comuns.
- **Não** são código pronto.
- São **estruturas reutilizáveis**.

De onde vêm:

- Experiência acumulada.
- Problemas repetidos.
- Generalização de soluções.

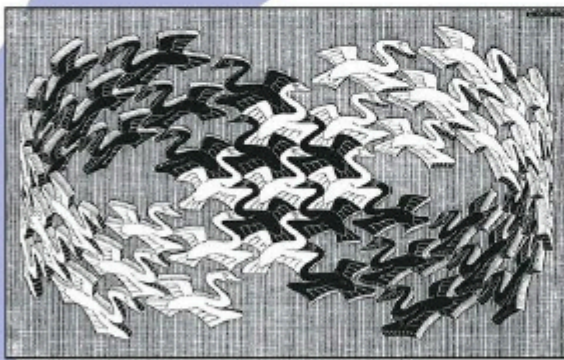
Cada padrão **descreve um problema recorrente**, e os padrões de projeto **compõem o MVC** ("podem compor").

5.2 O catálogo GoF

- **Livro clássico** (Gamma, Helm, Johnson, Vlissides; "[*Design Patterns: Elements of Reusable Object-Oriented Software*](#)").

Padrões de Projeto

Soluções reutilizáveis de software orientado a objetos



© 1994 M. C. Escher / London Art - Beum - Holland. Todos os direitos reservados.

ERICH GAMMA
RICHARD HELM
RALPH JOHNSON
JOHN VLISSIDES



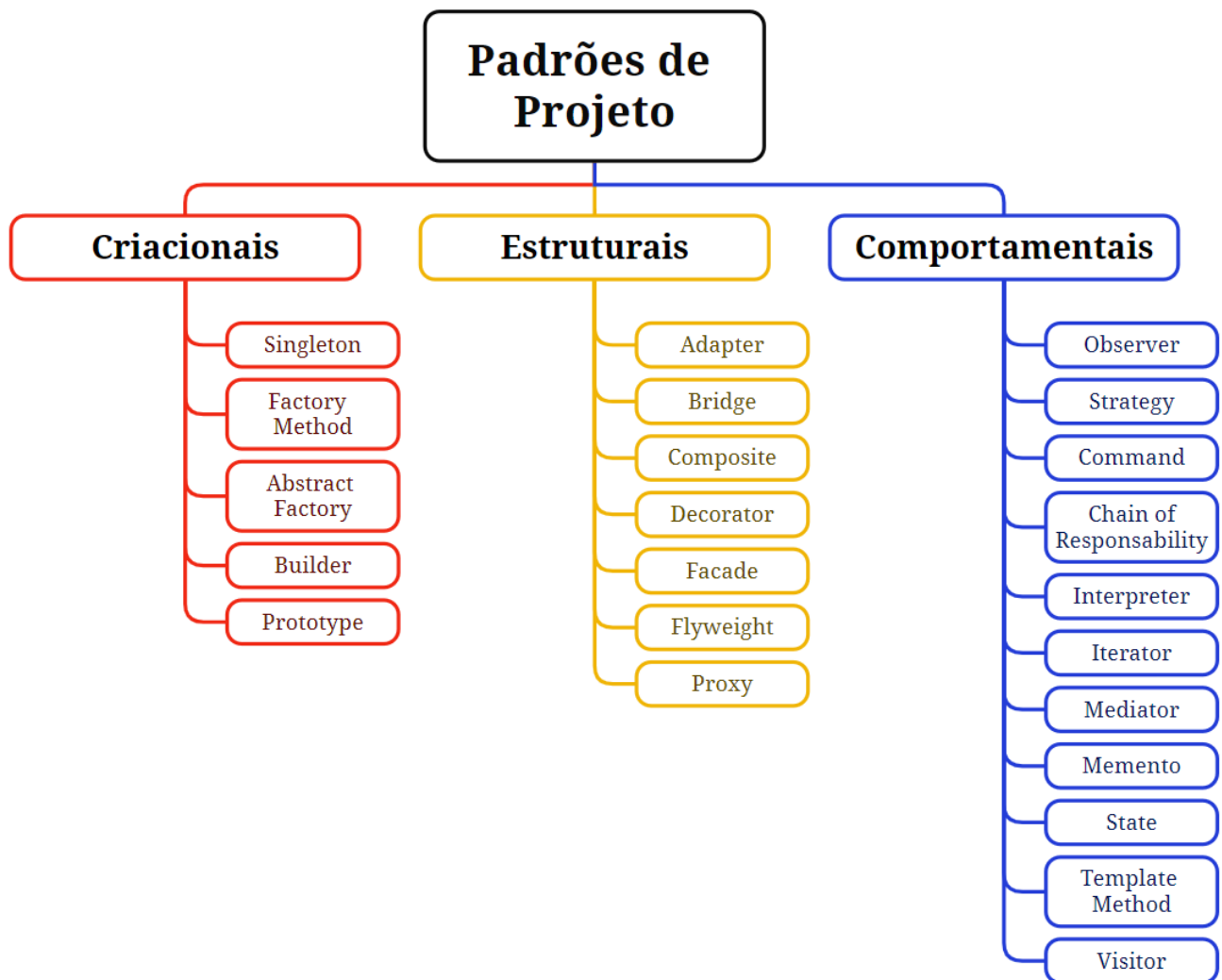
Design Patterns

- **23 padrões**, organizados por tipo.
- **Gang of Four** (ou "Gangue dos Quatro"), apelido dado aos quatro autores desse livro clássico
 - **Erich Gamma**
 - **Richard Helm**
 - **Ralph Johnson**
 - **John Vlissides**

Os três tipos

Tipo	Foco	O que fazem
Criacionais	Como os objetos são criados	Abstraem a criação de objetos, tornando o sistema menos dependente de classes concretas e mais flexível.
Estruturais	Como classes e objetos se organizam	Facilitam a composição de classes e objetos, definindo como combiná-los em estruturas maiores.
Comportamentais	Como os objetos se comunicam e colaboram	Definem algoritmos, fluxo de execução e responsabilidades entre objetos.

Principais padrões



Criacionais

- **Singleton:** garante uma única instância de uma classe.

- **Factory Method:** define uma interface para criar objetos, deixando a criação para as subclasses.
- **Abstract Factory:** cria famílias de objetos relacionados sem especificar suas classes concretas.
- **Builder:** constrói objetos complexos passo a passo.
- **Prototype:** cria novos objetos a partir da cópia de um objeto existente.

Estruturais

- **Adapter:** converte a interface de uma classe em outra interface esperada.
- **Bridge:** separa uma abstração da sua implementação, permitindo que variem independentemente.
- **Composite:** trata objetos individuais e composições de objetos da mesma forma (estruturas em árvore).
- **Decorator:** adiciona responsabilidades a um objeto dinamicamente.
- **Facade:** fornece uma interface simplificada para um subsistema complexo.
- **Proxy:** controla o acesso a um objeto, fornecendo um substituto.

Comportamentais

- **Strategy:** define uma família de algoritmos e torna os algoritmos intercambiáveis.
- **Observer:** define uma dependência um-para-muitos (um objeto notifica outros sobre mudanças).
- **Command:** encapsula uma requisição como um objeto, permitindo parametrizar, enfileirar e desfazer operações.
- **State:** permite que um objeto altere seu comportamento quando seu estado interno muda.
- **Template Method:** define o esqueleto de um algoritmo, deixando etapas específicas para as subclasses (hooks).
- **Iterator:** fornece uma maneira de percorrer os elementos de uma coleção sem expor sua estrutura interna.
- **Mediator:** centraliza a comunicação entre objetos, reduzindo o acoplamento.
- **Memento:** captura e restaura o estado interno de um objeto sem violar seu encapsulamento.

5.3 Quais padrões usar no MVC?

Observer (Model ↔ View), **Composite** (View) e **Strategy** (Controller).



O site **Refactoring Guru** (refactoring.guru/pt-br/design-patterns) explica cada padrão com exemplos e implementações em várias linguagens.

6. Integração geral

MVC = combinação de padrões:

Relação no MVC	Padrão	Classificação GoF
Model ↔ View	Observer	Comportamental
View	Composite	Estrutural
Controller	Strategy	Comportamental

7. Vantagens e desvantagens do uso de padrões

Vantagens:

- Promove **baixo acoplamento, reutilização e flexibilidade**.

Desvantagens:

- **Aumento da complexidade.**
- Pode ser **excessivo para sistemas simples** (overengineering: usar o padrão sem um problema real).

11. Considerações finais

- O MVC divide o sistema em **responsabilidades distintas**: **Model** cuida dos dados e das regras de negócio, **View** da apresentação e **Controller** da interação entre usuário e sistema. Essa separação **reduz o acoplamento** e facilita a evolução de cada parte.
- **Padrões de projeto** oferecem formas conhecidas de organizar classes e objetos para resolver problemas recorrentes. Ajudam a definir **responsabilidades, relacionamentos e pontos de variação**, tornando o código mais flexível e compreensível.
- A **arquitetura** define a organização do sistema em nível mais amplo. Nela, diferentes padrões podem ser combinados; assim, um sistema MVC pode empregar **Strategy, Observer, Composite e outros padrões** internamente.

Resumo

MVC COMO COMPOSIÇÃO DE PADRÕES

- |
- |— MVC
 - | |— Padrão arquitetural em camadas (Smalltalk; Django, Spring)
 - | |— Separa lógica de negócio da interface
 - | |— Resolve o acoplamento excessivo entre interface e lógica
- |
- |— COMPONENTES
 - | |— Model: lógica de negócio, dados e regras
 - | |— View: interface e apresentação
 - | |— Controller: recebe requisições, processa entrada, coordena Model e View
- |
- |— FLUXO
 - | |— Usuário → View → Controller → Model → (notifica) View → usuário
- |
- |— ESTUDO DE CASO: iFood
 - | |— Model: usuário, pedido, restaurante, entregador, pagamento
- |
- |— PADRÕES DE PROJETO (GoF: 23 padrões)
 - | |— Criacionais: criação de objetos
 - | |— Estruturais: organização de classes e objetos
 - | |— Comportamentais: comunicação e colaboração
- |
- |— PADRÕES DENTRO DO MVC
 - | |— Observer (Model ↔ View): Model = Subject, View = Observer
 - | |— Composite (View): hierarquia de componentes reutilizáveis
 - | |— Strategy (Controller): comportamento trocável sem mexer na View
- |
- |— BALANÇO
 - | |— Ganhos: baixo acoplamento, reutilização, flexibilidade
 - | |— Custo: mais complexidade (cuidado com overengineering)